

# A Generator–Realizer Architecture for Constrained Predictive Physical Human–Robot Interaction

**Draft status.** This document is a new research direction, not a revised presentation of the existing double-integrator MPC paper. The numerical claims in Sections 7 and 8 are generated by the code in this folder: Section 7 is a planar point-mass proof of concept, and Section 8 is the same generator–realizer architecture instantiated on a torque-controlled 7-DOF Franka FR3 in MuJoCo. The FR3 study is a focused core (impedance and admittance generators, horizon-wide torque feasibility, slack-relaxed workspace/speed bounds, a reactive-clipping comparator, one benchmark scenario) rather than the full validation program a manipulator-level claim would eventually need; human-participant, hardware, and comparisons against the broader predictive-impedance literature remain future work (Section 8.4).

## Abstract

Physical human–robot interaction controllers typically couple two decisions that are conceptually separate: what interaction behavior is desired, and how a constrained robot delivers it. This paper separates them architecturally. A pluggable interaction-dynamics generator specifies a desired contact-port acceleration  $a_k^{\text{id}} = f_\theta(e_k, \dot{e}_k, F_{h,k})$ ; a receding-horizon realization layer tracks that behavioral acceleration — not a motion trajectory — under actuator, command-rate, speed, and workspace constraints, and exposes the gap between what was requested and what the constrained robot delivered as an explicit realization residual. For the memoryless affine generator class, the decision variable, feasible set, and functional form of the quadratic program are fixed, while the generator changes the objective coefficients through  $(C_\theta, G_\theta)$ . We state this template-invariance property precisely and validate it with linear impedance and force-guided admittance.

In a reproducible planar simulation, the same predictive layer is used without retuning for both generators. When the unconstrained reference behavior would leave a 0.10 m workspace, instantaneous realization followed by command clipping reaches 0.266 m for the impedance generator and 0.525 m for the admittance generator. Predictive realization limits displacement to 0.100002 m and 0.0912 m, respectively, while respecting the 0.22 m/s component-wise speed bound to numerical tolerance. The required deviation from the desired dynamics is exposed as a measurable realization residual rather than hidden by parameter adaptation. A further experiment reassigns the

generator on one running controller instance — impedance, then admittance, then impedance — and verifies a rate-bounded transition without controller reconstruction.

We then instantiate the same architecture on a torque-controlled 7-DOF Franka FR3 in MuJoCo, freezing the task-space inertia, feedforward torque, and Jacobian at each solve (a local approximation, not exact horizon-wide prediction) and enforcing per-joint torque feasibility at every predicted step. Under a sustained 20 N push toward a 0.06 m/0.35 m/s component-wise workspace/speed bound, predictive realization holds displacement to within roughly 0.1--0.2 mm of the bound via a slack-relaxed soft constraint, while reactive clipping overshoots by 4.4 cm (impedance) and 4.7 cm (admittance). We report both the residual predicted by the local model and an empirical residual obtained from the MuJoCo end-effector velocity. These results establish a focused simulation proof of the architecture, not hardware safety or optimality of the separation.

**Keywords:** physical human–robot interaction, generator–realizer architecture, model predictive control, behavior-realization separation, impedance control, admittance control, constraint handling.

# 1. Introduction

A pHRI controller has two conceptually different responsibilities:

1. specify how the robot should move in response to contact; and
2. decide what motion can safely be executed by the physical robot.

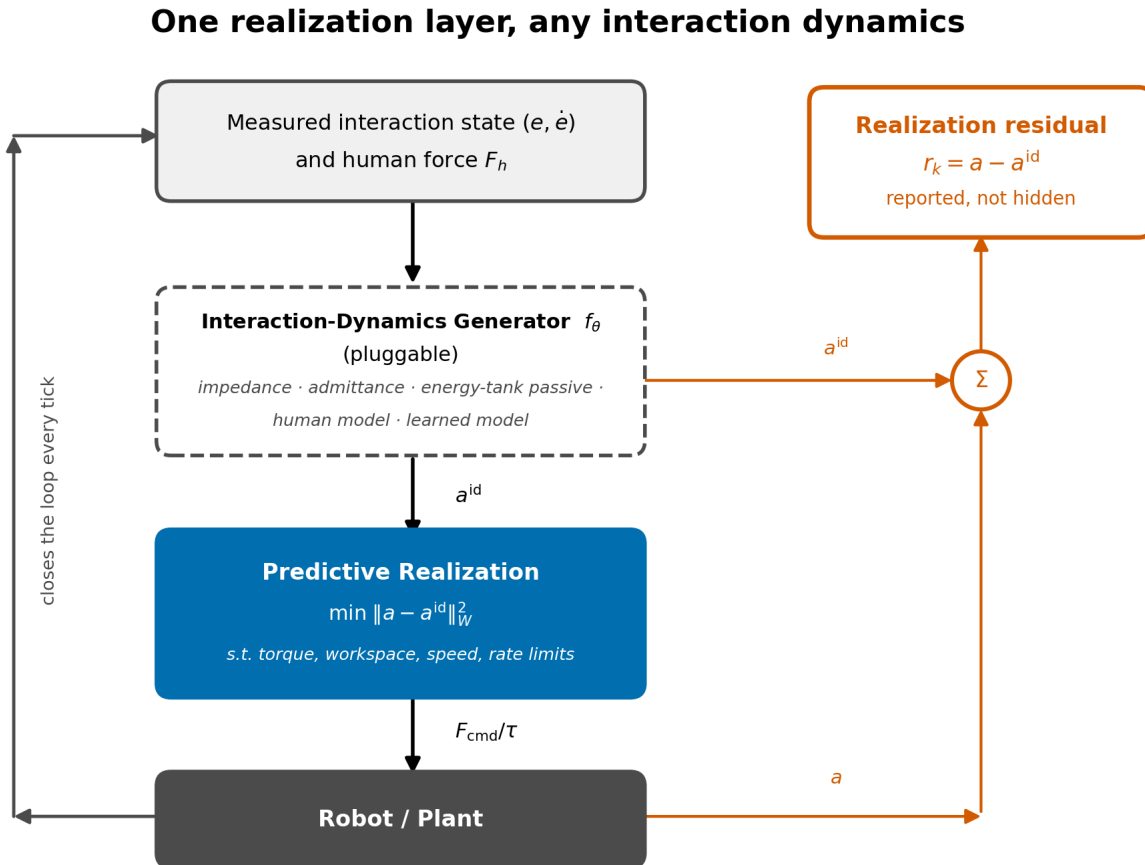
**Hypothesis.** For the memoryless affine generator class of Definition 1 (Section 3.3), interaction behavior can be supplied through a common desired-acceleration interface while the robot-level decision variable and feasible set remain unchanged. Theorem 1 proves this structural property. It does not claim that fixing the generator is lossless relative to jointly optimizing behavior parameters and robot commands.

Conventional impedance control combines these responsibilities in one mass–spring–damper feedback law. Variable-impedance methods add adaptation, and model-predictive variants can optimize impedance parameters or jointly plan motion and compliance. This is useful, but the behavior representation and the constraint-handling controller remain tied to impedance parameters.

A concrete consequence of that coupling is actuator saturation. A conventional impedance or PD law computes joint torque directly from the interaction error,  $\tau = J^\top(-K_d e - D_d \dot{e}) + \tau_{\text{ff}}$ , where  $\tau_{\text{ff}}$  is the feedforward gravity/Coriolis compensation and, on a redundant arm, the orientation-hold and null-space terms as well. Neither the impedance term nor  $\tau_{\text{ff}}$  reasons about the actuator's torque limit. A stiff enough impedance, a large enough interaction force, or simply an unfavorable configuration in which  $\tau_{\text{ff}}$  itself is large can drive the commanded torque past  $\tau_{\text{max}}$ . When the actuator clips, the

acceleration the impedance law intended is silently not the acceleration the robot delivers. This paper instead includes the *total frozen-model torque* — feedforward, orientation, null-space, and interaction terms — as a constraint at every prediction step (Section 8.1). Saturation is therefore anticipated within the local model, and the required deviation from the generator is represented by the realization residual. Because the manipulator prediction is frozen at each solve, this is not a global nonlinear-plant guarantee; executed torque is also monitored at the 1 kHz inner loop.

We study a different decomposition:



**Figure 1.** The generator (pluggable, dashed border) supplies a desired acceleration law  $a^{id}$ ; the realization layer (blue) retains one QP template and receives the generator-dependent law through its objective. The realization residual  $r_k = a - a^{id}$  (right) is reported explicitly rather than absorbed silently into the command.

The generator expresses the intended interaction. The predictive layer is not asked to choose stiffness or damping. It chooses the actual robot command, and it is permitted to depart from the reference dynamics only when required by physical constraints or secondary objectives. The magnitude and timing of that departure are explicit in the *realization residual*.

**This is what the hypothesis looks like as an architecture.** For affine generators, the realization layer retains one decision variable, feasible set, and objective template; the requested behavior enters

through  $(C_\theta, G_\theta)$ , which changes the numerical Hessian and gradient of the residual cost. Section 6 states this as the Affine Generator Template Invariance theorem. Section 4.1's impedance generator and Section 4.2's admittance generator are validated instances. Nonlinear, stateful, energy-tank, human-model, and learned generators retain the conceptual acceleration interface but may require linearization, state augmentation, additional constraints, or a nonlinear optimizer.

This separation changes both the algorithm and the scientific question. Instead of asking whether MPC can produce compliant tracking, we ask:

How accurately can a constrained robot realize a specified interaction dynamical system, and what controlled deviation is necessary when that system is temporarily incompatible with safety constraints?

The contributions of this draft are:

- a generator–realizer architecture whose robot decision variable, feasible set, and objective template are invariant across memoryless affine interaction generators, stated formally in Section 6;
- a convex predictive formulation for affine interaction dynamics, validated with impedance and force-guided admittance as two interchangeable instances of one generator interface;
- an explicit realization residual that quantifies how far the constrained robot had to depart from the specified dynamics, and why;
- a reproducible simulation, on both a planar point mass and a torque-controlled FR3 manipulator in MuJoCo, comparing horizon-wide constrained realization against instantaneous reference realization with identical command limits;
- an online generator-switching demonstration (Section 7.3) in which one controller instance accepts a different affine generator mid-run and the existing command-rate constraint bounds the transition.

We do **not** claim that embedding impedance behavior in MPC is itself new. Model predictive impedance control and model predictive interaction control already provide important prior formulations. The narrower contribution is an explicit desired-acceleration interface and realization residual together with template invariance for memoryless affine generators. Extension to nonlinear or stateful generators is future work, not covered by Theorem 1.

## 2. Related Work and Positioning

The closest literature is best distinguished by what the predictive or adaptive layer optimizes and by what quantity the robot-level controller is asked to reproduce.

| Class                              | Behavior representation                        | Optimized robot-level variable                          | Role of prediction                                           |
|------------------------------------|------------------------------------------------|---------------------------------------------------------|--------------------------------------------------------------|
| Predictive variable impedance      | $M_d, D_d, K_d$ , often with a trajectory      | Impedance parameters and/or trajectory                  | Select compliant behavior                                    |
| Reference-model adaptive impedance | $M_d\ddot{e} + D_d\dot{e} + K_de = F_h$        | Adaptive feedback parameters                            | Usually absent                                               |
| Direct robot MPC                   | State or tracking-error dynamics               | Force, torque, velocity, or acceleration                | Optimize robot motion directly                               |
| Impedance/interaction MPC          | Impedance, force, or coupled interaction model | Robot command and sometimes behavior parameters         | Embed interaction objectives and constraints                 |
| Reference governor                 | A single reference/setpoint signal             | Filtered reference fed to a fixed inner-loop controller | Supervise a non-predictive inner loop to enforce constraints |
| This work                          | $a^{\text{id}} = f_\theta(e, \dot{e}, F_h, z)$ | Robot command $u$                                       | Minimize dynamics-realization error under constraints        |

## 2.1 Predictive variable-impedance adaptation

Several approaches use MPC to adapt impedance parameters or to plan impedance and motion together. Anand et al. place a learned predictive policy above a low-level variable-impedance controller [1]. Haninger et al. predict interaction and optimize trajectory and impedance online while incorporating safety constraints [2]. Recent predictive variable-impedance formulations continue this parameter-adaptation direction [3]. Their optimized behavior variables are stiffness, damping, or related trajectory parameters.

## 2.2 Reference-model impedance control

Model-reference adaptive impedance control specifies desired impedance dynamics and designs an adaptive nonlinear controller so that the robot approaches them [4]. This establishes the value of separating a desired model from the physical robot, but it does not by itself provide horizon-wide handling of coupled state and input constraints.

## 2.3 Direct robot MPC

A separate class applies MPC directly to the robot or to a feedback-linearized error model. Its decision variables are robot commands rather than impedance parameters. Such a controller can generate an effective closed-loop impedance, but unless a desired interaction model appears explicitly in the prediction objective, that impedance is an induced property of the optimizer rather than the behavior it is asked to realize. This distinction separates the present formulation from our earlier double-integrator tracking MPC: the earlier controller penalizes interaction error, whereas the present controller penalizes disagreement with an independently specified interaction dynamical system.

## 2.4 Model predictive impedance and interaction control

Bednarczyk et al. formulate impedance behavior through MPC and handle practical constraints such as velocity, energy, and jerk [5]. Gold et al. formulate model predictive interaction control using robot and interaction models in the optimal-control problem and express manipulation objectives through costs and constraints [6]. Minniti et al. combine whole-body MPC with online adaptation of robot–environment interaction models for constrained mobile manipulation [7]. These papers are close precedents and make a broad “first constrained interaction-dynamics MPC” claim untenable.

## 2.5 Reference governors and supervisory constraint enforcement

Reference governors are the closest architectural relative to this paper's separation, from a different literature than Sections 2.1–2.4. A reference governor sits upstream of a fixed, already-designed inner-loop controller and modifies the reference signal so that constraints on the resulting closed-loop trajectory are respected [8]. The parallel is direct: a reference governor also separates what is commanded from what is safe to execute. The difference is where prediction lives and what is exchanged. A reference governor typically supervises a fixed inner loop and manipulates a reference signal; the realization layer here is itself predictive, and the exchanged object is an affine desired-acceleration law evaluated over predicted interaction states. The realization residual additionally measures the difference between requested and delivered dynamics. This connection limits the

novelty claim: the contribution is not separation in the abstract, but its formulation and measurement at the interaction-dynamics level.

## 2.6 Position of this work

Every row above except the last two optimizes a quantity tied to one behavior representation: impedance parameters, adaptive impedance gains, or robot-level tracking against a specific interaction model. The reference-governor row is architecturally closest. Our narrower distinction is that a memoryless affine desired interaction dynamics is exposed through a common acceleration interface, while the robot constraints are independent of the selected generator. This draft validates that interface for impedance and admittance. Demonstrating broader generality requires nonlinear/stateful generators and matched comparisons with reference governors, predictive impedance control, and model predictive interaction control.

## 3. Problem Formulation

### 3.1 General robot and contact-port dynamics

Consider a rigid manipulator

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau + J(q)^\top F_h,$$

where  $q \in \mathbb{R}^{n_q}$ ,  $\tau \in \mathbb{R}^{n_q}$ , and  $F_h$  is the human wrench applied at the interaction port. For a task coordinate  $y = h(q) \in \mathbb{R}^{n_y}$ ,

$$\dot{y} = J(q)\dot{q}, \quad \ddot{y} = J(q)\ddot{q} + \dot{J}(q, \dot{q})\dot{q}.$$

Substitution of the joint dynamics gives the affine acceleration map

$$\ddot{y} = b_y(q, \dot{q}, F_h) + G_y(q)\tau,$$

where

$$b_y(q, \dot{q}, F_h) = JM^{-1}(-C\dot{q} - g + J^\top F_h) + \dot{J}\dot{q},$$

$$G_y(q) = JM^{-1}.$$

Let  $y_d$  be a nominal interaction pose and define

$$e = y - y_d, \quad \dot{e} = \dot{y} - \dot{y}_d.$$

**Definition 1 (interaction-dynamics generator).** A generator is a causal map

$$\ddot{e}^{\text{id}} = f_{\theta}(e, \dot{e}, F_h, z),$$

where  $z$  denotes optional internal generator state and  $\theta$  denotes behavior parameters. Its output is a desired contact-port acceleration, not a robot command. The generator is deliberately unaware of robot torque limits, joint limits, or workspace geometry; those belong to the realization layer.

The proposed architecture does not ask the generator to compute  $\tau$ . Instead, it supplies  $\ddot{e}^{\text{id}}$  above, and the predictive realization layer selects  $\tau$  so that the physically realized  $\ddot{e}$  approaches  $\ddot{e}^{\text{id}}$  while satisfying robot constraints.

The full manipulator formulation is the target architecture. The executable study in this draft uses the following exactly discretized point-mass specialization so that the generator–realizer distinction can be isolated without kinematic or inverse-dynamics confounds.

## 3.2 Planar proof-of-concept plant

The proof-of-concept plant is a planar point mass:

$$m_r \ddot{p}(t) = u(t) + F_h(t),$$

where  $p \in \mathbb{R}^2$  is displacement from a nominal interaction pose,  $u \in \mathbb{R}^2$  is commanded robot force,  $F_h \in \mathbb{R}^2$  is measured human force, and  $m_r > 0$  is the realized robot mass. Define

$$x = \begin{bmatrix} p^\top & v^\top \end{bmatrix}^\top, \quad v = \dot{p}.$$

Under zero-order hold with period  $\Delta t$ ,

$$x_{k+1} = Ax_k + B(u_k + F_{h,k}),$$

$$A = \begin{bmatrix} I & \Delta t I \\ 0 & I \end{bmatrix}, \quad B = \begin{bmatrix} \frac{\Delta t^2}{2m_r} I \\ \frac{\Delta t}{m_r} I \end{bmatrix}.$$

The actual acceleration is

$$a_k = \frac{u_k + F_{h,k}}{m_r}.$$

## 3.3 The generator specialized to the planar plant

For the planar plant of Section 3.2, Definition 1 specializes with  $x_k = [p_k^\top \ v_k^\top]^\top$  in place of  $(e, \dot{e})$ :

$$a_k^{\text{id}} = f_{\theta}(x_k, F_{h,k}, z_k).$$

For the affine class considered here,



$$a_k^{\text{id}} = C_\theta x_k + G_\theta F_{h,k}.$$

### 3.4 Realization residual

We define

$$r_k = a_k - a_k^{\text{id}}.$$

If  $r_k = 0$ , the robot exactly realizes the requested interaction dynamics at that sample. If  $r_k \neq 0$ , the robot executes the closest feasible dynamics selected by the optimizer. Reporting  $r_k$  distinguishes unavoidable safety intervention from failure to tune the reference model.

This definition is intentionally stronger than commanding a nominal impedance force and penalizing

$$\left\| u_k - u_k^{\text{imp}} \right\|_2^2.$$

The latter is controller-output tracking: it penalizes deviation from a nominal force command.  $\|r_k\|_W^2$  is tracking too, but of a categorically different target — the *behavioral acceleration* the generator specifies, not a prior controller's output — so it compares the realized physical acceleration directly with the requested interaction dynamics. It remains meaningful when the generator is admittance-based, nonlinear, energy-based, or learned and no  $u_k^{\text{imp}}$  exists.

## 4. Configurable Generators

Section 3.1's Definition 1 specifies the generator interface abstractly; Section 3.3 specialized it to the planar plant. This section instantiates it. The two validated affine rows below use the same QP template by supplying  $(C_\theta, G_\theta)$ . The remaining rows share only the desired-acceleration interface: their state, nonlinearity, and uncertainty may require augmentation, linearization, additional constraints, or a nonlinear realization problem. They are included as a scoped research roadmap, not as already supported drop-in generators.

| Generator                     | Requested dynamics                                     | What changes in the QP                       | Status                    |
|-------------------------------|--------------------------------------------------------|----------------------------------------------|---------------------------|
| Impedance (4.1)               | $M_d a^{\text{id}} + D_d v + K_d p = F_h$              | $C_\theta, G_\theta$ only                    | Validated, Sections 7–8   |
| Force-guided admittance (4.2) | $T_a a^{\text{id}} + v = Y F_h$                        | $C_\theta, G_\theta$ only                    | Validated, Sections 7–8   |
| Energy-tank passive dynamics  | $a^{\text{id}} = M_d^{-1}(F_h - D(v)v - \nabla V(p)),$ | May require generator-state augmentation and | Sketched, not implemented |

| Generator                         | Requested dynamics                                  | What changes in the QP                                          | Status                    |
|-----------------------------------|-----------------------------------------------------|-----------------------------------------------------------------|---------------------------|
| (4.3)                             | tank-modulated                                      | linearization or NMPC                                           |                           |
| Human internal/intent model (4.3) | $a^{\text{id}}$ from an estimated human control law | May require estimator-state prediction and uncertainty handling | Sketched, not implemented |
| Learned interaction model (4.3)   | $a^{\text{id}}$ from a fitted or trained model      | May require nonlinear prediction and robustness constraints     | Sketched, not implemented |

The first two rows use the same Section 5 affine QP template. The remaining rows share the desired-acceleration interface only; whether they retain a QP depends on their state, nonlinearity, and uncertainty structure.

## 4.1 Impedance generator

The desired impedance is

$$M_d a^{\text{id}} + D_d v + K_d p = F_h.$$

For isotropic parameters,

$$a^{\text{id}} = -\frac{K_d}{M_d} p - \frac{D_d}{M_d} v + \frac{1}{M_d} F_h.$$

Thus

$$C_{\text{imp}} = \begin{bmatrix} -K_d M_d^{-1} I & -D_d M_d^{-1} I \end{bmatrix}, \quad G_{\text{imp}} = M_d^{-1} I.$$

Unlike variable-impedance MPC (Section 2.1),  $M_d$ ,  $D_d$ ,  $K_d$  are not decision variables in the present formulation. They describe the requested behavior.

## 4.2 Force-guided admittance generator

The second generator requests a force-dependent velocity:

$$T_a a^{\text{id}} + v = Y F_h,$$

or

$$a^{\text{id}} = -T_a^{-1} v + T_a^{-1} Y F_h.$$

It has no position-restoring term. After force release, velocity decays and the displaced position is retained. This provides a qualitatively different behavior through the same acceleration interface.

## 4.3 Future nonlinear generators

The three remaining rows of Section 4's catalog table are not variants of one equation; each replaces the affine law differently. A nonlinear passive dynamics, with a general (not necessarily linear) damping term and a potential function in place of a linear spring,

$$a^{\text{id}} = M_d^{-1} (F_h - D(v)v - \nabla V(p)),$$

is one instance. An energy-tank-modulated version adds a tank state that bounds the energy the generator may inject. A human internal/intent model or a learned interaction model instead replaces  $f_\theta$  with an estimator or trained model. These extensions preserve the desired-acceleration interface but may require state augmentation, sequential linearization, nonlinear MPC, uncertainty propagation, and additional constraints. They are not covered by the affine QP theorem.

## 5. Predictive Realization

Section 4 supplies  $a^{\text{id}}$ . For a memoryless affine generator it is represented by  $C_\theta, G_\theta$ , producing one condensed-QP template whose numerical cost coefficients change with the generator while its decision variable and feasible set do not. At time  $k$ , let

$$U_k = \begin{bmatrix} u_{0|k}^\top & \cdots & u_{N-1|k}^\top \end{bmatrix}^\top.$$

The controller solves

$$\begin{aligned} \min_{U_k} \quad & \sum_{i=0}^{N-1} \|a_{i|k} - a_{i|k}^{\text{id}}\|_W^2 + \lambda_u \|u_{i|k}\|_2^2 + \lambda_{\Delta u} \|u_{i|k} - u_{i-1|k}\|_2^2 \\ \text{s.t.} \quad & x_{i+1|k} = Ax_{i|k} + B(u_{i|k} + \hat{F}_{h,i|k}), \\ & a_{i|k} = (u_{i|k} + \hat{F}_{h,i|k})/m_r, \\ & a_{i|k}^{\text{id}} = C_\theta x_{i|k} + G_\theta \hat{F}_{h,i|k}, \\ & |u_{i|k}|_\infty \leq u_{\max}, \\ & |u_{i|k} - u_{i-1|k}|_\infty \leq \dot{u}_{\max} \Delta t, \\ & |p_{i+1|k}|_\infty \leq p_{\max}, \\ & |v_{i+1|k}|_\infty \leq v_{\max}. \end{aligned}$$

Only  $u_{0|k}^*$  is executed.  $u_{-1|k}$  in the  $i = 0$  rate penalty and rate constraint denotes the command actually applied at the previous solve, carried by the controller between calls (zero before the first solve), so both are well-defined from the very first call. The implementation uses the measured force held constant across the horizon,  $\hat{F}_{h,i|k} = F_{h,k}$ , rather than oracle knowledge of the scripted force. A learned or physics-based force predictor can be substituted without changing the generator interface.

## 5.1 Tracking behavioral acceleration, not a motion trajectory

The objective  $\|a_{i|k} - a_{i|k}^{\text{id}}\|_W^2$  is tracking — of a categorically different target than a position or velocity trajectory. No desired position trajectory is constructed; the optimizer instead compares two accelerations evaluated at the predicted interaction state: the acceleration the constrained robot will produce and the behavioral acceleration the interaction model requests. Position and velocity enter as generator states and as safety-constrained quantities, never as a tracked reference in their own right.

## 5.2 The decision variable is the robot command, not the generator's parameters

The optimization variable is the robot command sequence  $U_k$ . The generator parameters  $\theta$  are fixed during each experiment. Switching from impedance to admittance changes  $C_\theta, G_\theta$ , not the QP constraints or the command variable — stated formally as Theorem 1 (Section 6).

# 6. Basic Properties

Section 1 asserted, and Section 5.2 restated, that swapping the generator changes nothing in the QP except  $(C_\theta, G_\theta)$ . This section states that claim as a theorem and proves it, then establishes what the resulting QP guarantees in principle — convexity, exact realization when unconstrained, one-step constraint satisfaction — and, Proposition 3 in particular, what it does not.

**Theorem 1 (Affine Generator Template Invariance).** Fix the plant  $(A, B, m_r)$ , horizon  $N$ , weights  $(W, \lambda_u, \lambda_{\Delta u})$ , and bounds  $(u_{\max}, \dot{u}_{\max}, p_{\max}, v_{\max})$ . There is a single map  $(C, G) \mapsto Q(C, G)$  — a quadratic-program template depending only on the plant, horizon, weights, and bounds above, never on  $\theta$  — such that, for every generator  $\theta$  with affine law  $(C_\theta, G_\theta)$ , the realization QP of Section 5 is exactly  $Q(C_\theta, G_\theta)$ . Consequently, for any two generators  $\theta_1, \theta_2$ , the instances  $Q(C_{\theta_1}, G_{\theta_1})$  and  $Q(C_{\theta_2}, G_{\theta_2})$  share the same decision variable  $U_k$  and the same feasible set, and differ only in the numerical objective coefficients contributed by  $(C_\theta, G_\theta)$ , not in the decision-variable dimensions, constraint set, or objective template. Realizing a different affine generator therefore changes only the supplied  $(C_\theta, G_\theta)$ . This is stated for the affine generator class of Definition 1; a nonlinear generator

(Section 4.3) makes  $a_{i|k}^{\text{id}}$  a nonlinear function of  $U_k$ , so  $\|r_{i|k}\|_W^2$  is no longer guaranteed convex quadratic and the template  $Q$  does not apply without per-step linearization.

*Proof.* Every constraint in Section 5's QP — the dynamics recursion, the acceleration definition, and the four box constraints — is written in terms of  $A, B, m_r, u_{\max}, \dot{u}_{\max}, p_{\max}, v_{\max}, U_k$ , and the resulting state/acceleration sequence; none references  $\theta, C_\theta$ , or  $G_\theta$ . The feasible set is therefore a fixed polyhedron, identical for every generator. The only appearance of  $(C_\theta, G_\theta)$  anywhere in the problem is the term  $a_{i|k}^{\text{id}} = C_\theta x_{i|k} + G_\theta \hat{F}_{h,i|k}$  inside the primary cost, entering exactly once per horizon step as an affine substitution into  $r_{i|k} = a_{i|k} - a_{i|k}^{\text{id}}$ . Because  $x_{i|k}$  is itself independent of  $(C_\theta, G_\theta)$  and affine in  $U_k$  (established in Proposition 1's proof below),  $a_{i|k}^{\text{id}}$  is affine in  $U_k$  with  $(C_\theta, G_\theta)$ -dependent coefficients only, so  $\|r_{i|k}\|_W^2$  is a convex quadratic in  $U_k$  whose Hessian and gradient contributions are determined entirely by  $(C_\theta, G_\theta)$ ; the secondary terms  $\lambda_u \|u_{i|k}\|_2^2$  and  $\lambda_{\Delta u} \|u_{i|k} - u_{i-1|k}\|_2^2$  do not involve  $(C_\theta, G_\theta)$  at all. Collecting these pieces defines  $Q(C, G)$  as a function of  $(C, G)$  alone — the plant, horizon, weights, and bounds enter  $Q$  as fixed parameters, not arguments — and by construction  $Q(C_\theta, G_\theta)$  is exactly the QP Section 5 poses for generator  $\theta$ . ■

This is why Section 4's five-instance catalog is checkable rather than aspirational: each affine row supplies a different  $(C_\theta, G_\theta)$  into the one template  $Q$  above, and the theorem is precisely the statement that nothing else needs to change. The nonlinear rows of Section 4.3 are future work for exactly the reason the theorem's affine hypothesis makes explicit: without per-step linearization,  $f_\theta$  does not factor through a fixed  $(C, G)$  pair, and  $Q$  does not apply.

**Proposition 1 (convexity for affine generators).** If  $W \succeq 0$ ,  $\lambda_u \geq 0$ , and  $\lambda_{\Delta u} \geq 0$ , then the finite-horizon problem in Section 5 is a convex quadratic program. It is strictly convex if the assembled Hessian is positive definite, including the common case  $\lambda_u > 0$ .

*Proof.* The lifted state  $x_{i|k}$  is affine in  $U_k$  by the recursion  $x_{i+1|k} = Ax_{i|k} + B(u_{i|k} + \hat{F}_{h,i|k})$  unrolled from  $x_{0|k}$ . Hence  $a_{i|k} = (u_{i|k} + \hat{F}_{h,i|k})/m_r$  and  $a_{i|k}^{\text{id}} = C_\theta x_{i|k} + G_\theta \hat{F}_{h,i|k}$  are affine in  $U_k$ , so  $r_{i|k} = a_{i|k} - a_{i|k}^{\text{id}}$  is affine in  $U_k$  and  $\|r_{i|k}\|_W^2$  is convex quadratic since  $W \succeq 0$ . The regularization terms  $\lambda_u \|u_{i|k}\|_2^2$  and  $\lambda_{\Delta u} \|u_{i|k} - u_{i-1|k}\|_2^2$  are convex quadratic in  $U_k$  for  $\lambda_u, \lambda_{\Delta u} \geq 0$ , and a sum of convex functions is convex. Every listed constraint bounds an affine function of  $U_k$  in absolute value, so the feasible set is an intersection of half-spaces, hence a convex polyhedron. ■

**Proposition 2 (exact unconstrained realization).** Assume additionally that  $W \succ 0$  (strictly positive definite — a stronger hypothesis than Proposition 1's  $W \succeq 0$ ). Suppose an input sequence exists for which  $r_{i|k} = 0$  over the horizon and no constraint is active. If the realization residual is optimized lexicographically before secondary effort objectives—or equivalently the secondary weights are zero—then every optimal primary solution realizes the reference dynamics exactly.

*Proof.* The primary objective  $\sum_i \|r_{i|k}\|_W^2$  is nonnegative and attains the value zero at the assumed feasible sequence, so its minimum over the horizon is zero. Under lexicographic priority (or zero secondary weights), any optimal solution must attain this minimum, so  $\sum_i \|r_{i|k}\|_W^2 = 0$ , and since each term is nonnegative,  $\|r_{i|k}\|_W^2 = 0$  for every  $i$ . Because  $W \succ 0$ , a quadratic form  $\|r\|_W^2$  vanishes only at  $r = 0$ ; if  $W$  were only positive semi-definite, this step would give  $r_{i|k} \in \ker W$  rather than  $r_{i|k} = 0$ , which is why the stronger hypothesis is needed here but not in Proposition 1. ■

With finite scalar weights, effort regularization can intentionally introduce a small residual even when exact realization is feasible. The present experiment uses small, not zero, secondary weights, so the predictive controller's measured residual is not exactly zero even during unconstrained intervals (Table 1) — this is the finite-weight case Proposition 2 itself distinguishes from the idealized zero-weight one, not a violation of it. The reactive comparator applies the generator's instantaneous law directly, with no horizon and no secondary regularization to trade against; its residual sits at numerical-solver tolerance (Table 1) for the simpler reason that it is solving a different, unconstrained algebraic inversion, not the same QP with different weights. That near-exactness is a check that the underlying generator and dynamics model are implemented correctly, not a demonstration of Proposition 2 itself.

**Proposition 3 (one-step constraint inheritance).** Assume the model and current human force are exact over the executed sample and the QP is feasible. Then  $u_{0|k}^*$ ,  $p_{1|k}$ , and  $v_{1|k}$  satisfy their corresponding QP bounds. Re-solving at every sample yields constraint satisfaction by induction while feasibility is retained.

This proposition is not a recursive-feasibility theorem. A deployable version requires a terminal invariant set, soft constraints with a quantified fallback, or a backup safe controller. Sudden force changes inside one sample and model error must also be covered by robust constraint tightening.

## 7. Simulation

Section 6 establishes what the QP guarantees in principle — convexity, exact realization when unconstrained, one-step constraint satisfaction, and what it does not guarantee (recursive feasibility). This section tests those properties empirically on the planar plant of Section 3.2, comparing predictive realization against the reactive comparator under identical command and rate limits.

### 7.1 Reproducible setup

The simulation uses a 2.5 kg planar point mass,  $\Delta t = 0.02$  s, and a 20-step (0.4 s) horizon. A smooth 12 N force is applied along  $+y$  from 1 s to 3 s. Limits are

$$|u_j| \leq 18 \text{ N}, \quad |v_j| \leq 0.22 \text{ m/s}, \quad |p_j| \leq 0.10 \text{ m},$$

with  $|\dot{u}_j| \leq 180$  N/s. The realization, force, and force-rate weights are  $1$ ,  $2 \times 10^{-4}$ , and  $10^{-3}$ , respectively.

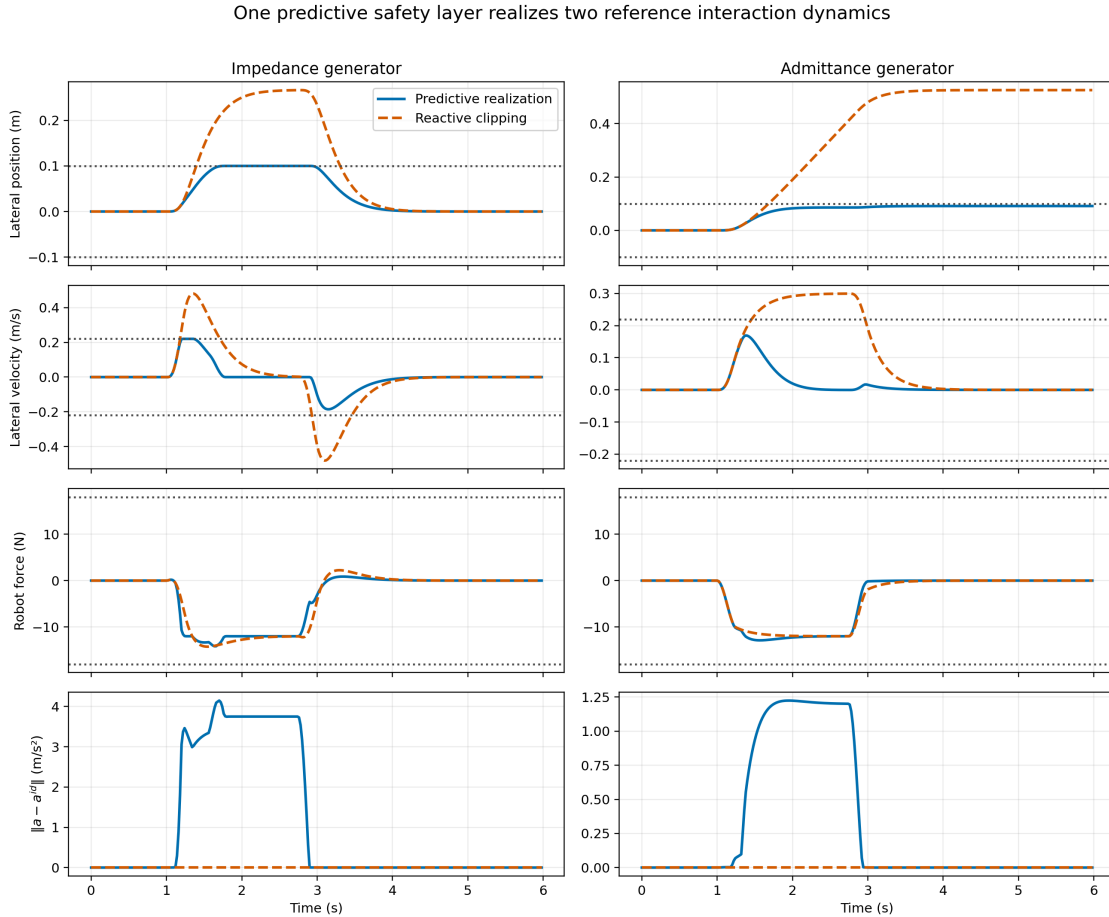
The impedance generator (Section 4.1) uses

$$M_d = 2.0 \text{ kg}, \quad D_d = 18 \text{ Ns/m}, \quad K_d = 45 \text{ N/m}.$$

The admittance generator (Section 4.2) uses  $T_a = 0.25$  s and  $Y = 0.025$  m/(Ns).

The comparator computes the instantaneous reference acceleration, converts it to robot force, and applies the same 18 N force and 180 N/s slew limits. It has no prediction of position or speed constraints. This comparator is intentionally minimal: it isolates what horizon-wide state constraints change; it is not presented as a survey-complete baseline.

## 7.2 Results



**Figure 2.** The same predictive realization layer is used with impedance (left) and force-guided admittance (right). Dotted lines are component-wise limits. Reactive clipping follows the desired dynamics almost exactly but does not anticipate state-limit violations. Predictive realization departs from the generator when necessary, visible in the final row.

| Generator  | Controller                | Realization<br>RMSE (m/s <sup>2</sup> ) | Peak $ p_j $<br>(m) | Peak<br>speed<br>(m/s) | Peak<br>$ u_j $<br>(N) | State-limit<br>violation             |
|------------|---------------------------|-----------------------------------------|---------------------|------------------------|------------------------|--------------------------------------|
| Impedance  | Predictive<br>realization | 1.351                                   | 0.100002            | 0.220                  | 14.16                  | 0.000002 m<br>numerical<br>tolerance |
| Impedance  | Reactive<br>clipping      | $< 10^{-6}$                             | 0.2663              | 0.480                  | 14.24                  | 0.1663 m,<br>0.260 m/s               |
| Admittance | Predictive<br>realization | 0.405                                   | 0.0912              | 0.169                  | 12.88                  | none                                 |
| Admittance | Reactive<br>clipping      | $< 10^{-6}$                             | 0.5250              | 0.300                  | 12.00                  | 0.4250 m,<br>0.080 m/s               |

**Table 1.** State-limit violation reports the amount by which the 0.10 m / 0.22 m/s workspace/speed bound is exceeded. Realization RMSE for the reactive comparator is at numerical-solver tolerance because it directly inverts the generator's instantaneous law for the force that realizes it exactly, with no horizon or secondary regularization to trade against; it is not zero for predictive realization because the horizon-wide formulation trades a small residual for constraint satisfaction and finite secondary weights (Proposition 2, Section 6), and because the reported metric aggregates over the whole run, not only the constrained interval.

The impedance reference has a static displacement  $F_h/K_d = 12/45 = 0.267$  m, matching the 0.266 m simulated peak after the smooth force ramp. Because this behavior is incompatible with the 0.10 m workspace, the predictive controller increases the realization residual while the bound is active. After release, it returns to the impedance equilibrium at the origin.

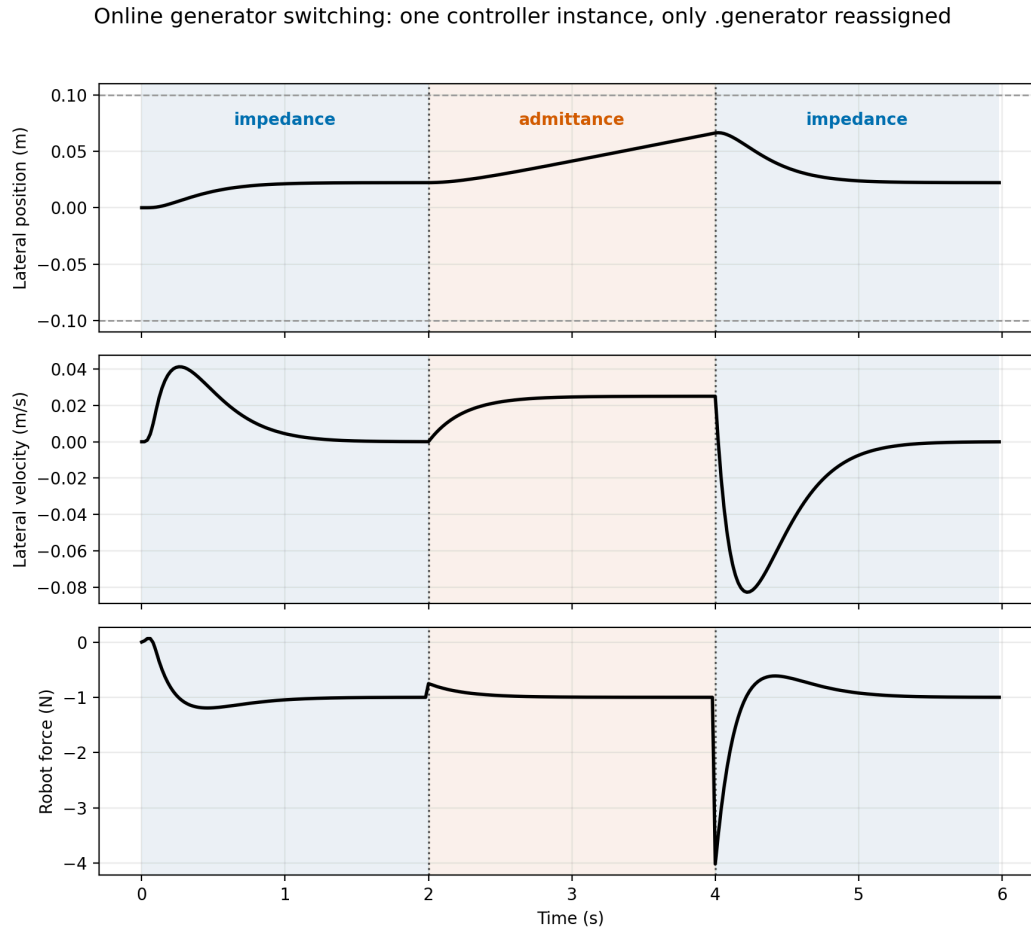
The admittance reference requests approximately 0.30 m/s under 12 N and has no restoring spring. Reactive realization therefore accumulates 0.525 m displacement and retains it after release. The constrained controller begins departing from the requested velocity dynamics before reaching the workspace boundary and settles at 0.091 m. The generator remains unchanged; the behavior modification is entirely attributable to the realization layer.

## 7.3 Online generator switching

Sections 7.1–7.2 validate the generator interface by running two separate simulations, one per generator — informative, but not yet a demonstration that a single running controller accepts a different generator without redesign. This section tests that directly. One `InteractionDynamicsMPC`



instance is constructed once, with the impedance generator; a small constant lateral force (1 N, well inside every bound) is applied for the entire 6 s run, with no release. At  $t = 2$  s and  $t = 4$  s, exactly one attribute is reassigned on the live controller — `controller.generator` — switching impedance  $\rightarrow$  admittance  $\rightarrow$  impedance; nothing else (the QP weights, constraints, decision variable, or the `previous_command` state carried between solves) is touched or reconstructed.



**Figure 3.** One controller instance; only `.generator` is reassigned at  $t = 2$  s and  $t = 4$  s (dotted lines). Position converges to the impedance equilibrium  $F_h/K_d \approx 0.022$  m during both impedance segments, drifts at the admittance generator's steady-state velocity  $YF_h = 0.025$  m/s with no restoring term during the middle segment, and re-converges to the same equilibrium after re-entering impedance from a different state (nonzero position and velocity) at  $t = 4$  s.

Both switches are rate-bounded: the largest command-force jump at a switch instant is 3.02 N at  $t = 4$  s, 84% of the QP's 3.6 N per-tick limit. This is a material command change, not evidence of mathematical smoothness; it simply satisfies the same discrete rate constraint that bounds every other step. Position and speed stay within 0.067 m and 0.083 m of the origin throughout, well inside the 0.10 m / 0.22 m/s bounds. The experiment therefore validates the software-level generator interface and feasibility of an online swap under the existing rate constraint; it does not establish optimal switching, hybrid stability, or continuity of the generator law. The second impedance segment

additionally shows re-convergence from the different position and nonzero velocity left by the admittance segment without special-casing on re-entry.

## 7.4 What the experiments establish

The results support four limited claims:

1. two qualitatively different affine interaction models use the same constrained predictive implementation;
2. an explicit dynamics residual identifies when safety requires deviation from the requested behavior;
3. horizon-wide state constraints prevent violations that command clipping alone cannot prevent in this deterministic model;
4. a single controller instance accepts a different generator online, mid-run, by reassigning one attribute, with its command change bounded by the existing rate constraint.

They do not establish manipulator-level feasibility, coupled human–robot stability, passivity, robustness to force-estimation delay, or real-time performance on embedded hardware.

## 8. FR3 Manipulator Validation

Section 7 isolates the generator–realizer distinction from kinematic and inverse-dynamics confounds by using a point mass. This section instantiates the same architecture on a torque-controlled 7-DOF Franka FR3 simulated in MuJoCo, replacing the exact double-integrator plant with the real, configuration-dependent manipulator dynamics of Section 3.1.

### 8.1 Architecture

At each solve, let  $J_v(q) \in \mathbb{R}^{3 \times 7}$  be the translational Jacobian and  $\Lambda(q)^{-1} = J_v M(q)^{-1} J_v^\top$  the (regularized) inverse operational-space inertia for the position coordinate.

**Full closed-loop law.** Let  $R_d$  be the held reference orientation,  $e_R(q) \in \mathbb{R}^3$  the corresponding orientation error,  $J_w(q) \in \mathbb{R}^{3 \times 7}$  the rotational Jacobian, and  $q_0$  a fixed neutral configuration. The commanded joint torque is assembled from four terms:

$$\begin{aligned}
\tau_{\text{ff}}(q, \dot{q}) &= C(q, \dot{q})\dot{q} + g(q) && \text{(feedforward)} \\
\tau_{\text{orient}}(q, \dot{q}) &= J_w(q)^\top (-K_{\text{rot}} e_R(q) - D_{\text{rot}} \omega) && \text{(orientation hold, raw)} \\
\tau_{\text{null}}(q, \dot{q}) &= -k_{\text{null}}(q - q_0) - d_{\text{null}}\dot{q} && \text{(null-space centering, raw)} \\
\bar{N}_v(q) &= I_7 - M(q)^{-1} J_v(q)^\top \Lambda(q) J_v(q) && \text{(position-consistent null-space projector)} \\
\tau_{\text{aux}}(q, \dot{q}) &= \bar{N}_v(q)^\top [\tau_{\text{orient}}(q, \dot{q}) + \tau_{\text{null}}(q, \dot{q})] && \text{(projected, see below)} \\
\tau_{\text{base}}(q, \dot{q}) &= \tau_{\text{ff}}(q, \dot{q}) + \tau_{\text{aux}}(q, \dot{q})
\end{aligned}$$

and the executed torque is

$$\tau = \tau_{\text{base}}(q, \dot{q}) + J_v(q)^\top F_{\text{cmd}}.$$

Task acceleration obeys  $\ddot{y} = J_v \ddot{q} + \dot{J}_v \dot{q}$ . Consequently, define

$$d_{\text{known}}(q, \dot{q}) = J_v(q) M(q)^{-1} \tau_{\text{aux}}(q, \dot{q}) + \dot{J}_v(q, \dot{q}) \dot{q},$$

which contains both the projected auxiliary-torque contribution and the task-kinematic term. The residual translational dynamics used by the QP are

$$\ddot{e} = \Lambda(q)^{-1} (F_{\text{cmd}} + F_h) + d_{\text{known}},$$

which corrects the sign of the sketch in the earlier draft of this section: the realized acceleration is *positive* in the commanded force, matching the point-mass convention of Section 3.2.  $F_{\text{cmd}} \in \mathbb{R}^3$ , the QP's Cartesian correction force, is supplied by whichever controller is active. The predictive controller sets  $F_{\text{cmd}} = F_{0|k}^*$ , the first element of the receding-horizon solution (Section 5, condensed later in this section). The reactive comparator instead sets

$$F_{\text{cmd}} = \text{clip} \left( \Lambda(q) [a^{\text{id}}(x, F_h) - d_{\text{known}}] - F_h \right),$$

rate- and magnitude-limited to the same bounds as the predictive controller, with  $a^{\text{id}} = C_\theta x + G_\theta F_h$  the generator's instantaneous desired acceleration (Section 4). Both controllers share the identical  $\tau_{\text{base}}$  and the identical torque-assembly step; they differ only in how  $F_{\text{cmd}}$  is chosen.

$\bar{N}_v(q)$  above is not the only choice that projects the auxiliary torque away from the translation task, and getting this projection right mattered in practice. A first implementation projected the null-space torque through the null-space projector of the *full* 6-row Jacobian and left the orientation-hold torque unprojected. Both choices leaked into  $\ddot{e}$ . Under sustained interaction this produced a positive-feedback path: joint drift increased the auxiliary torque, which increased translational leakage and the compensating command. Applying the dynamically consistent position-task projector to both auxiliary terms removes that structural path [9]. For an exact inverse,  $J_v M^{-1} \bar{N}_v^\top = 0$ . The implementation uses Tikhonov regularization when forming  $\Lambda$ , however, so orthogonality is

approximate rather than exact; the remaining projected-torque leakage is retained explicitly in  $d_{\text{known}}$ , together with  $\dot{J}_v \dot{q}$ .

This remaining leakage is small at any single tick but one-directional, and a weak null-space centering spring lets it accumulate: over a full multi-second run it can drive the joint configuration more than a radian from  $q_0$  before the reported benchmark numbers below were corrected, which in turn grows  $d_{\text{known}}$  further and inflates the correction the controller must apply — the same feedback structure as the leakage path above, just slow enough that it is invisible on a short test and only shows up over the full benchmark duration. Confirmed empirically by sweeping  $k_{\text{null}}$ ,  $d_{\text{null}}$  and tracking configuration deviation over the full run rather than an early window:  $k_{\text{null}} = 10$ ,  $d_{\text{null}} = 2$  allows deviation past 1.3 rad in three of the four benchmark conditions;  $k_{\text{null}} = 40$ ,  $d_{\text{null}} = 8$ , the gentlest increase found that controls it, keeps deviation under roughly 0.2 rad throughout, without measurably damping either generator's own requested response (checked directly). Stronger gains still (e.g. 100/20) control the drift further but visibly suppress the admittance generator's task-space response, which is why the gentler value is used.

The realization QP freezes  $\Lambda(q)^{-1}$ ,  $d_{\text{known}}(q, \dot{q})$ ,  $\tau_{\text{base}}(q, \dot{q})$ , and  $J_v(q)$  at the current solve and holds them across the horizon. In compact form, its decision vector contains the Cartesian command sequence and nonnegative state slacks,

$$\begin{aligned} \min_{\{F_i, s_i^p, s_i^v\}_{i=0}^{N-1}} \quad & \sum_{i=0}^{N-1} \left( \|\ddot{e}_i - a_i^{\text{id}}\|_W^2 + \lambda_F \|F_i\|_2^2 + \lambda_{\Delta F} \|F_i - F_{i-1}\|_2^2 + \rho_p \|s_i^p\|_1 + \rho_v \|s_i^v\|_1 \right), \\ & \ddot{e}_i = \Lambda_k^{-1}(F_i + \hat{F}_{h,i}) + d_{\text{known},k}, \\ & |F_i| \leq F_{\text{max}}, \quad |F_i - F_{i-1}| \leq \Delta F_{\text{max}}, \\ & |\tau_{\text{base},k} + J_{v,k}^\top F_i| \leq \tau_{\text{max}}, \\ & |e_i| \leq e_{\text{max}} + s_i^p, \quad |\dot{e}_i| \leq v_{\text{max}} + s_i^v, \quad s_i^p, s_i^v \geq 0. \end{aligned}$$

This per-solve model is a convex QP but only a local predictor of the nonlinear MuJoCo plant. The hard torque constraint is imposed at every predicted step, rather than only  $i = 0$ . It guarantees feasibility of the frozen-model command sequence returned by a successful solve; it is not a horizon-wide guarantee for the future nonlinear trajectory. Executed torque is therefore recomputed and monitored at 1 kHz.

Note that this bound is on the *sum*  $\tau_{\text{base}} + J_v^\top F_{\text{cmd},i}$ , not on  $F_{\text{cmd},i}$  alone:  $\tau_{\text{base}}$  is not a fixed, negligible offset, and in an unfavorable configuration (near a joint limit, or during an orientation-hold transient) it can itself consume a substantial share of  $\tau_{\text{max}}$ , leaving less budget for the interaction correction than a constraint on  $F_{\text{cmd},i}$  alone would assume. This is precisely the actuator-saturation risk motivated in Section 1: the constraint here is written against the total predicted torque so that

saturation is a modeled, anticipated event rather than something the actuator's own clipping discovers after the fact. Concretely, the bound is part of the QP's feasible set, not a check applied to its output: any  $F_{\text{cmd}}$  sequence the solver returns satisfies it by construction, because an infeasible sequence is not a candidate solution in the first place. This is the mechanistic difference from a PD or impedance law, which evaluates  $\tau = J^\top(-K_d e - D_d \dot{e}) + \tau_{\text{ff}}$  directly and hands it to the actuator; if that value exceeds  $\tau_{\text{max}}$ , something downstream truncates it, and the acceleration the law intended silently stops being the acceleration delivered. Here, when the generator's desired acceleration would require more torque than the budget allows, the optimizer instead returns the closest feasible  $F_{\text{cmd}}$  — trading realization accuracy against feasibility — and that trade-off is exactly the realization residual  $r_k$  of Section 3.4: reported, not hidden.

The Cartesian workspace and speed box is slack-relaxed with a large penalty ( $10^8$ ). Under the frozen-Jacobian approximation, a hard box has no recursive-feasibility guarantee for the true nonlinear dynamics. The slack penalty keeps the optimization defined when the local prediction cannot satisfy the box exactly while making violation costly. If a solve is reported infeasible, the implementation falls back to the rate- and magnitude-limited reactive law. No fallback is triggered in the reported benchmark.

The fast control loop recomputes  $\tau_{\text{base}}$ , the orientation term, and the null-space term from the current  $(q, \dot{q})$  at 1 kHz; the QP is re-solved only every 20 ms (a 50 Hz outer loop), holding  $F_{\text{cmd}}$  between solves. Caching the full joint torque at the outer-loop rate, rather than only the slow correction term, would confound any comparison of predictive against reactive control at a common inner-loop rate.

## 8.2 Benchmark scenario

Mirroring the planar prototype's own story rather than a separate benchmark design: the EE holds a fixed nominal pose and orientation while a sustained human push toward a workspace/speed boundary is applied, and predictive realization is compared against reactive clipping under an identical command/rate box. Neither controller has oracle knowledge of the scripted force; both receive a zero-order-hold forecast of the currently measured force, as in Section 7.

The push is a 20 N force along  $-z$  with a raised-cosine ramp from  $t = 1.0$  s to  $1.25$  s, a hold to  $t = 3.25$  s, and a symmetric ramp-down to  $t = 3.5$  s. It is applied physically to the MuJoCo end-effector body, not only supplied to the controller's internal model, so the comparison reflects what the real simulated dynamics do under the push, not only what each controller believes is happening. The 2 s hold (versus an earlier 0.5 s) gives the admittance generator's velocity time to build toward its steady state (time constant  $T_a = 0.3$  s below); at the shorter hold, the corrected dynamics above show the reactive comparator does not approach the workspace bound at all, so no genuine predictive-versus-reactive contrast exists to report for that generator. The impedance generator's story is unaffected by

the hold length, since its peak is set by its equilibrium, reached well within 0.5 s (confirmed directly at both hold values).

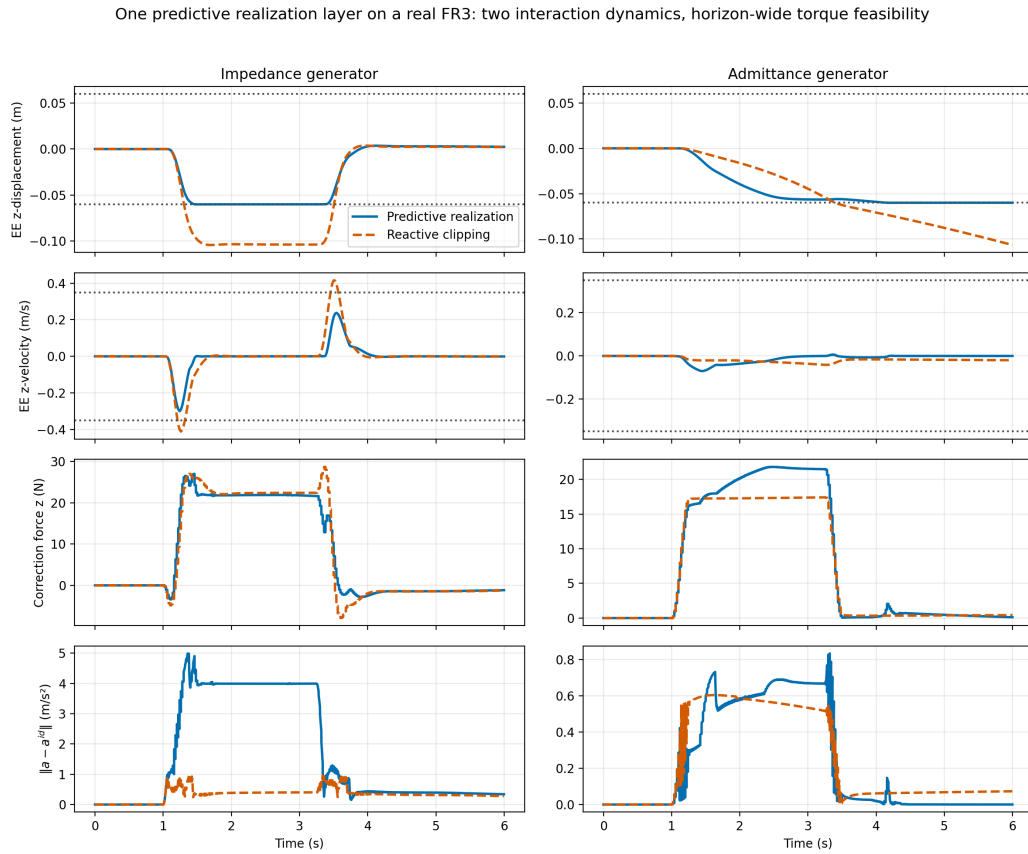
The workspace/speed bound is

$$|e_z| \leq 0.06 \text{ m}, \quad |v| \leq 0.35 \text{ m/s},$$

the QP horizon is 15 steps (0.3 s) at  $\Delta t = 0.02$  s, and per-joint torque limits match the FR3 ( $\pm 87$  Nm for joints 1–4,  $\pm 12$  Nm for joints 5–7).

The impedance generator (Section 4.1) uses  $M_d = 2.0$  kg,  $D_d = 28$  Ns/m,  $K_d = 200$  N/m. The admittance generator (Section 4.2) uses  $T_a = 0.3$  s and  $Y = 0.01$  m/(Ns). Each condition runs for 6 s of simulated time.

## 8.3 Results



**Figure 4.** The same predictive realization layer on a real FR3, impedance (left) and admittance (right) generators, predictive realization (solid) against reactive clipping (dashed). Dotted lines are the workspace/speed bounds. Predictive realization tracks the bound closely for the duration of the push; reactive clipping overshoots substantially, and for the admittance generator does not recover the displacement after force release.

| Generator  | Controller                | Empirical /<br>predicted<br>residual<br>RMSE<br>(m/s <sup>2</sup> ) | Peak<br>$ e_z $<br>(m) | Peak<br>component<br>speed (m/s) | Maximum<br>torque<br>utilization | State-<br>limit<br>violation |
|------------|---------------------------|---------------------------------------------------------------------|------------------------|----------------------------------|----------------------------------|------------------------------|
| Impedance  | Predictive<br>realization | 2.41 / 2.50                                                         | 0.0602                 | 0.300                            | 37.1%                            | 0.0002 m                     |
| Impedance  | Reactive<br>clipping      | 0.387 /<br>0.107                                                    | 0.1044                 | 0.415                            | 37.2%                            | 0.0444<br>m, 0.065<br>m/s    |
| Admittance | Predictive<br>realization | 0.367 /<br>0.502                                                    | 0.0601                 | 0.070                            | 34.4%                            | 0.0001 m                     |
| Admittance | Reactive<br>clipping      | 0.346 /<br>0.059                                                    | 0.1066                 | 0.041                            | 31.7%                            | 0.0466 m                     |

**Table 2.** The *predicted* residual uses the frozen local model in Section 8.1. The *empirical* residual instead finite-differences the 1 kHz MuJoCo end-effector velocity and compares that acceleration with the generator request; it is the appropriate measure of behavior delivered by the simulated nonlinear plant. Their gap measures local-model and differentiation error, so the predicted residual must not be interpreted as an exact plant residual. Speed is reported component-wise to match the box constraint. Maximum torque utilization is the largest ratio  $|\tau_j|/\tau_{\max,j}$ , which avoids comparing unlike 87 Nm and 12 Nm joint limits. Predictive solve times are reported in the text below because the reactive comparator does not solve a QP.

No condition violates a per-joint torque limit at the executed sample, no solve is reported infeasible, and the torque constraint is never the one that actively binds in this benchmark: maximum torque utilization stays below 38% in every condition, well short of saturation. This benchmark does not include a first-step-only ablation, so it does not by itself isolate the performance benefit of enforcing that constraint over the full horizon rather than only at the first step; that benefit is established structurally (Section 8.1) and by the horizon-wide feasibility check reported directly (Section 8.1), not by this benchmark exercising the bound.

The impedance reference has an unconstrained static displacement  $F_h/K_d = 20/200 = 0.10$  m, which by itself already exceeds the 0.06 m bound. Reactive clipping tracks toward that equilibrium and peaks at 0.104 m (slightly above the static value, from the same ramp-overshoot effect as the

planar case in Section 7.2), overshooting the bound by 4.4 cm. Predictive realization instead departs from the requested dynamics while the bound is active, holding displacement to 0.0602 m.

The admittance reference has no equilibrium displacement to compare against, only a steady-state velocity  $Y F_h = 0.01 \times 20 = 0.2$  m/s while the force is held, with no restoring term to pull it back after release. Reactive clipping accumulates displacement through the hold and continues briefly after force release before the velocity decays (time constant  $T_a = 0.3$  s), peaking at 0.1066 m — a 4.7 cm overshoot — and never recovers it, since the generator itself has no position-restoring term (Section 4.2) and the reactive comparator has no lookahead to anticipate the boundary. Predictive realization begins departing from the requested velocity before the bound is reached and stays within 0.0601 m.

One limitation is visible in the results and worth reporting rather than hiding: the admittance-generator QP's solve time (mean 10.3 ms, max 70.7 ms in this run) is within the nominal 20 ms outer-loop period on average but exceeds it at its worst tick; the impedance generator's solves (mean 4.4 ms, max 7.5 ms) do not approach it at all. Both use the same dimensions and constraint structure but different generator-dependent objective coefficients. Conditioning is a plausible explanation for the difference, but it is not isolated experimentally. The loop is simulation-time paced, so these numbers indicate solver effort rather than a real-time guarantee.

An earlier version of this benchmark, run before the null-space centering fix described in Section 8.1, showed a second issue: slack chattering over a late-time window as the admittance condition sat near the workspace bound. That was traced to the same slow configuration drift Section 8.1 now describes and controls; it is not present in the results above (Figure 4), and is not reported as a current limitation.

## 8.4 Scope and what remains

This is a focused core, not the full manipulator validation program. The generator set is impedance and admittance only, not the nonlinear/passive generator of Section 4.3; the scenario is a single sustained push, not a sweep over delay, sensor noise, or mass mismatch; there is no collision constraint, and the reactive clipped comparator is intentionally weak. A publication-level evaluation needs matched classical impedance/admittance, control-barrier or reference-governor, predictive-variable-impedance, model-predictive-impedance, and model-predictive-interaction baselines. The online generator-switching demonstration (Section 7.3) is planar only; an FR3 counterpart remains future work.

Human-participant and hardware validation remain future work, as does closing the admittance solve-time gap identified in Section 8.3.



## 9. Limitations and Open Theory

The results above establish the architectural distinction on two plants; they do not establish everything a mature interaction-dynamics realization framework would need. Five gaps are worth stating plainly rather than leaving implicit.

First, the current generator class is affine and memoryless (Section 4.3 sketches richer instances, none yet implemented). A stateful or learned generator requires augmented prediction and a clear treatment of model uncertainty.

Second, the realization cost does not itself imply passivity. A passive reference generator can still be rendered non-passive by constraint-induced deviation or secondary optimization. A useful extension is a dissipativity constraint on the realized port variables together with an energy tank or passivity observer/controller.

Third, hard state constraints can become infeasible after an unpredicted impulse — Proposition 3 already flags this as a gap, not a guarantee. The FR3 study (Section 8) addresses it operationally with a slack-relaxed workspace/speed box and a reactive fallback on solver infeasibility, and both were exercised without producing unbounded violations in the tested scenario — but this is an empirical demonstration for one scenario, not the terminal invariant set, quantified soft-constraint fallback, or backup safe controller Proposition 3 says a deployable version would need. Softening safety-critical constraints without defining the resulting risk remains not sufficient on its own.

Fourth, the interaction-dynamics residual has physical units and depends on the chosen output coordinates, and the FR3 study only exercises it on the translational coordinate. Orientation on the FR3 is held by a fixed PD law,  $\tau_{\text{orient}}$  (Section 8.1), not by a generator. The architectural claim is therefore validated for translation only. Extending the interface to orientation requires a principled metric or energy-weighted normalization across coordinates with different units. Even within one coordinate, raw RMSE values across generators with different characteristic scales are not direct performance rankings. In addition, Table 2's empirical and frozen-model residuals differ materially in several conditions. That discrepancy is evidence that model adequacy and acceleration estimation must be reported separately, not a solver residual or a violation of Theorem 1.

Finally, the architectural novelty must be assessed against the full model predictive impedance/interaction-control literature surveyed in Section 2, not only the two closest precedents named there. The strongest eventual claim should rest on implemented generator modularity, formal residual guarantees, and matched experiments—not terminology alone.

# 10. Conclusion

This paper's central result is architectural: for memoryless affine generators, Theorem 1 establishes invariance of the QP template, decision-variable dimensions, and feasible set while  $(C_\theta, G_\theta)$  changes the numerical residual-cost coefficients. The simulations instantiate that claim for impedance and admittance. The planar study shows anticipatory state-constraint handling and an online generator swap whose command change respects the existing rate bound. The FR3 study shows the same interface on a nonlinear manipulator with frozen-model horizon constraints; it enforces joint torque feasibility at every horizon step and limits workspace overshoot relative to a reactive clipped comparator, though the torque bound is never the one that actively binds in this particular benchmark (Section 8.3). It also exposes an important qualification: the residual predicted by the local operational-space model can differ substantially from the residual measured from MuJoCo motion. The present evidence is therefore a reproducible simulation proof of concept, not a claim of universal generator compatibility, passivity, hardware safety, or superiority to established predictive interaction controllers. The next steps are a genuinely nonlinear or learned generator, stronger matched baselines, model-mismatch and delay sweeps, a first-step-versus-horizon torque-constraint ablation, and hardware or participant validation.

# References

1. A. Anand et al., “Deep Model Predictive Variable Impedance Control,” 2022. [arXiv:2209.09614](#).
2. K. Haninger, C. Hegeler, and L. Peternel, “Model Predictive Impedance Control with Gaussian Processes for Human and Environment Interaction,” *Robotics and Autonomous Systems*, vol. 165, 104431, 2023. DOI: [10.1016/j.robot.2023.104431](#).
3. S. Xue et al., “Model predictive variable impedance control towards safe robotic interaction in unknown disturbance-rich environments,” *Robotics and Autonomous Systems*, 2025. [Publisher page](#).
4. M. Sharifi, S. Behzadipour, and G. Vossoughi, “Nonlinear model reference adaptive impedance control for human–robot interactions,” *Control Engineering Practice*, 2014. [Publisher page](#).
5. M. Bednarczyk et al., “Model Predictive Impedance Control,” *IEEE International Conference on Robotics and Automation*, 2020. DOI: [10.1109/ICRA40945.2020.9196969](#).
6. T. Gold, A. Völz, and K. Graichen, “Model Predictive Interaction Control for Robotic Manipulation Tasks,” *IEEE Transactions on Robotics*, 2023. DOI: [10.1109/TRO.2022.3196607](#).
7. M. V. Minniti, R. Grandia, K. Fähr, F. Farshidian, and M. Hutter, “Model Predictive Robot-Environment Interaction Control for Mobile Manipulation Tasks,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2021. [arXiv:2106.04202](#).

8. E. Garone, S. Di Cairano, and I. Kolmanovsky, “Reference and command governors for systems with constraints: A survey on theory and applications,” *Automatica*, vol. 75, pp. 306–328, 2017. DOI: [10.1016/j.automatica.2016.08.013](https://doi.org/10.1016/j.automatica.2016.08.013).
9. O. Khatib, “A Unified Approach for Motion and Force Control of Robot Manipulators: The Operational Space Formulation,” *IEEE Journal on Robotics and Automation*, vol. 3, no. 1, pp. 43–53, 1987. DOI: [10.1109/JRA.1987.1087068](https://doi.org/10.1109/JRA.1987.1087068).

## Reproducibility

Run from pHRI/imp\_reference :

```
# Architecture schematic (Figure 1)
python3 simulation/make_architecture_diagram.py

# Planar point-mass study (Section 7, Figure 2)
python3 simulation/run_experiments.py
python3 -m pytest simulation/test_interaction_dynamics_mpc.py simulation/test_benchmark_

# Online generator-switching demonstration (Section 7.3, Figure 3)
python3 simulation/run_generator_switching_experiment.py
python3 -m pytest simulation/test_generator_switching.py -q

# FR3/MuJoCo manipulator study (Section 8, Figure 4)
python3 simulation/run_fr3_experiments.py
python3 -m pytest simulation/test_fr3_interaction_dynamics_mpc.py simulation/test_fr3_benchmark_
```

Both driver scripts use a fixed deterministic scenario and record every numeric parameter and reported metric in `results/metrics.json` and `results/fr3_metrics.json`, respectively, but neither asserts anything itself. The `test_interaction_dynamics_mpc.py` / `test_fr3_interaction_dynamics_mpc.py` suites check the QP's mechanics on individual, hand-picked states; they do not run the 6-second scripted-force scenario. Tables 1 and 2 report numbers from. `test_benchmark_verification.py` / `test_fr3_benchmark_verification.py` close that gap: they run the actual benchmark scenario end-to-end and assert its headline claims directly — no torque-limit violation in any condition, no unhandled solver infeasibility, predictive realization within a controlled envelope of the workspace bound, and reactive clipping overshooting it by a checked margin — so a future change that silently breaks a claim in this paper fails a test rather than only showing up as a quieter number in a table nobody re-checks. Solve-time claims are bounded generously rather than asserted exactly, since they are wall-clock and were confirmed empirically to vary run-to-run (the position/torque numbers, by contrast, are exactly reproducible: re-running both

benchmarks produced bit-for-bit identical values before this paper's figures and tables were finalized). The FR3 study imports the shared MuJoCo/operational-space infrastructure from `pHRI/simulation` ( `fr3_mujoco.py` , `fr3_impedance.py` , `so3_utils.py` ) rather than duplicating it.